

Command Palette

Icon Loading Fix Guide

Fixing icon-caused extension crashes:
wrong API, missing Build Action, and bad file paths

ISSUE REPORTED

Extension loads fine without icon.

Adding any icon causes it to stop loading entirely

EXTENSION

DiskAnalyzerExtension 1.2.0.0

ROOT CAUSE

New IconInfo(path) + wrong BuildAction

REPORT DATE

July 14, 2026

VOLUME

Vol. 3 of 3

Three Independent Root Causes

Extension-with-icon crashes are almost always a combination of all three causes below. Each one alone can stop the extension loading — together they guarantee it.

1 Wrong API — new `IconInfo()` used for file paths

CRITICAL

`new IconInfo(path)` only accepts Unicode glyphs, emojis, or remote URLs. Passing a local file path like `"Assets\\disk.png"` throws silently, killing the COM host process before any UI appears.

2 Wrong Build Action — icon file excluded from package

CRITICAL

PNG files added to a project default to Build Action: None. This means the file is never copied into the deployed MSIX package. The extension starts, tries to load the icon, finds no file, throws an `IOException`, and the host kills it silently.

3 Missing `Package.appxmanifest` reference

HIGH

The MSIX packager only bundles files that are Content or referenced in the manifest. Even with Build Action: Content, `StoreLogo.png` must exist and be referenced in the manifest Properties block, or the extension tile shows a broken image and `CmdPal` may skip loading the extension.

! All Three Often Present Simultaneously

The scaffolded template sets Build Action: None for all PNGs, and many tutorials show `new IconInfo(path)` incorrectly. Fix all three causes at once.

Fix 1 — Use `IconHelpers.FromRelativePath()` for File Icons

The correct API for local PNG icons is `IconHelpers.FromRelativePath()` from the `Microsoft.CommandPalette.Extensions.Toolkit` NuGet package. It resolves the path relative to the package's installed location at runtime — the only place the file actually exists after deploy.

API Comparison

```
// =====
// new IconInfo() – what it actually accepts
// =====
Icon = new IconInfo("Assets\\disk.png"); // ■ local path – CRASHES
Icon = new IconInfo("C:\\Dev\\...\\disk.png"); // ■ abs path – CRASHES
Icon = new IconInfo("disk.png"); // ■ bare name – CRASHES

Icon = new IconInfo("\uE9F9"); // ■ Segoe glyph – always works
Icon = new IconInfo("■"); // ■ emoji – always works
Icon = new IconInfo("https://..."); // ■ remote URL – always works

// =====
// IconHelpers.FromRelativePath() – correct for local PNG files
// =====
using Microsoft.CommandPalette.Extensions.Toolkit;

Icon = IconHelpers.FromRelativePath("Assets\\disk.png"); // ■ correct
```

Apply It in Your Page and Commands

```
public sealed class DiskAnalyzerPage : ListPage
{
    public DiskAnalyzerPage()
    {
        Name = "Disk Analyzer";
        // ■ File-based icon – resolved at runtime from package location
        Icon = IconHelpers.FromRelativePath("Assets\\disk.png");
    }
}

public sealed class OpenDriveCommand : InvokableCommand
{

```

Fix 2 — Set Build Action: Content on All Icon Files

Every PNG in your Assets/ folder must be marked **Content** so the MSIX packager includes it. The default when you add a PNG to a project is **None** — meaning it is silently excluded from the deployed package.

Fix in Solution Explorer (Visual Studio)

Step	Navigation / Action
1	In Solution Explorer, expand the Assets/ folder
2	Click your icon file (e.g. disk.png)
3	Press F4 to open the Properties pane
4	Set Build Action → Content
5	Set Copy to Output Directory → Copy if newer
6	Repeat for every PNG in Assets/

Fix in .csproj (Edit Project File)

Right-click project → **Edit Project File**. Find every `<None Include="Assets\...">` entry and replace with `<Content>`:

```
<!-- ■ WRONG – default for PNGs added to project -->
<None Include="Assets\disk.png" />
<None Include="Assets\drive.png" />

<!-- ■ CORRECT – Content is required for MSIX packaging -->
<ItemGroup>
  <Content Include="Assets\disk.png">
    <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
  </Content>
  <Content Include="Assets\drive.png">
    <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
  </Content>
  <Content Include="Assets\StoreLogo.png">
    <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
  </Content>
  <Content Include="Assets\Square44x44.png">
    <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
  </Content>
  <Content Include="Assets\Square150x150.png">
    <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
  </Content>
</ItemGroup>
```

Fix 3 — Verify Package.appxmanifest References

The manifest must reference the required icon sizes. Open **Package.appxmanifest** → **Visual Assets** tab in the designer (or view XML source directly). Verify the <Properties> and <Applications> blocks include valid icon paths.

Minimum Required Manifest Entries

```
<Properties>
  <Logo>Assets\StoreLogo.png</Logo>  <!-- 50x50 — CmdPal tile -->
</Properties>

<Applications>
  <Application ...>
    <uap:VisualElements
      Square150x150Logo="Assets\Square150x150.png"
      Square44x44Logo="Assets\Square44x44.png"
      Description="Analyze disk usage from Command Palette"
      BackgroundColor="transparent">
    </uap:VisualElements>
  </Application>
</Applications>
```

Required Asset Sizes

File	Size	Used For	Required?
Assets\StoreLogo.png	50 × 50 px	CmdPal extension tile icon	YES
Assets\Square44x44.png	44 × 44 px	Taskbar / app icon	YES
Assets\Square150x150.png	150 × 150 px	Start Menu tile	YES
Assets\disk.png	Any (256 rec)	Your custom extension icon	No — custom
Assets\drive.png	Any (256 rec)	Per-item list icon (optional)	No — custom

Complete Correct Project Structure

Here is the full folder layout and which files are required vs. optional:

- DiskAnalyzerExtension/
 - **DiskAnalyzerExtension.csproj** ← add <Content> entries
 - **Package.appxmanifest** ← verify icon references
 - Program.cs
 - DiskAnalyzerExtension.cs ← IExtension
 - **DiskAnalyzerPage.cs** ← use IconHelpers here
 - Assets/
 - **StoreLogo.png** ← 50x50 – REQUIRED
 - **Square44x44.png** ← 44x44 – REQUIRED
 - **Square150x150.png** ← 150x150 – REQUIRED
 - disk.png ← your custom icon
 - drive.png ← per-item icon (opt)

SECTION 6

Fix 4 — Safe Icon Fallback Pattern

Wrap icon loading so that a missing or corrupt file never crashes the extension. This is especially important during development when assets are still changing:

```
// Always-safe icon helper – falls back to a glyph if file missing
private static IconInfo SafeIcon(
    string relativePath,
    string fallbackGlyph = "\uE9F9")
{
    try
    {
        var icon = IconHelpers.FromRelativePath(relativePath);
        return icon ?? new IconInfo(fallbackGlyph);
    }
    catch
    {
        return new IconInfo(fallbackGlyph); // glyph: zero file I/O
    }
}

// Usage – safe in constructor, safe on list items
Icon = SafeIcon("Assets\\disk.png", "\uE9F9");
Icon = SafeIcon("Assets\\drive.png", "\uEDA2");
```

★ **Fastest Fix If You Just Want It Working Now**

Swap to a Segoe Fluent glyph: `Icon = new IconInfo("\uE9F9");` — it requires zero files, never fails, and looks native in CmdPal. Then add file-based icons once the extension is fully stable.

i **Useful Segoe Fluent Icon Glyphs for Disk Extensions**

`\uE9F9` = Storage | `\uEDA2` = Hard Drive | `\uE838` = Folder | `\uE8B7` = Chart | `\uE9F5` = SD Card | `\uE7C3` = Page | `\uEC4A` = Pie Chart. Full reference:

learn.microsoft.com/windows/apps/design/style/segoe-fluent-icons-font

Complete Fix Checklist

Work through all items — each one independently can cause the no-load crash:

■ Replace all new <code>IconInfo("Assets\\...")</code> with <code>IconHelpers.FromRelativePath("Assets\\...")</code>
■ Add using <code>Microsoft.CommandPalette.Extensions.Toolkit</code> ; at top of each file that uses icons
■ In Solution Explorer: set every PNG Build Action → Content (press F4 to open Properties)
■ In Solution Explorer: set every PNG Copy to Output Directory → Copy if newer
■ In .csproj: change every to
■ Verify <code>Assets\\StoreLogo.png</code> exists and is exactly 50 × 50 px PNG
■ Verify <code>Assets\\Square44x44.png</code> exists (44 × 44 px)
■ Verify <code>Assets\\Square150x150.png</code> exists (150 × 150 px)
■ In <code>Package.appxmanifest</code> : confirm <code>Assets\\StoreLogo.png</code> in
■ Wrap all <code>IconHelpers.FromRelativePath()</code> calls in try/catch with glyph fallback
■ Right-click project → Deploy to re-package with the updated Content files
■ Open <code>CmdPal</code> → type 'reload' → Reload Command Palette extensions
■ Verify extension appears AND icon renders correctly
■ Check logs at <code>%LOCALAPPDATA%\Microsoft\PowerToys\CmdPal\Logs</code> for any remaining errors

COMPLETE SERIES SUMMARY

Three-Volume Fix Series — Full Overview

Vol.	PDF Title	Fixes Covered
1	DiskAnalyzer_Fix_Guide.pdf	Async I/O; constructor crash; <code>Task.Run()</code> ; <code>RaiseItemsChanged()</code> ; try/catch; de
2	CmdPal_Navigation_Fix_Guide.pdf	<code>CommandResult</code> types; replacing <code>NoOpCommand</code> ; <code>GoHome</code> / <code>GoBack</code> / <code>Dis</code>
3	CmdPal_Icon_Fix_Guide.pdf ← THIS FILE	<code>IconHelpers.FromRelativePath()</code> ; Build Action: Content; manifest references;